K. R. MANGALAM UNIVERSITY

B. TECH CSE (Data Science)

SOET

(School of Engineering &Technology)

ENCS256-ANALYSIS AND DESIGN OF ALGORITHMS

CAPSTONE ASSIGNMENT

Submitted by-

Name - Sakshi Jain

Roll No. – 2401420034

Submitted to-

Aarti Ma'am

1. Setup

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
import itertools
import random
import time
```

2. Delivery Network

Code:

```
graph = {
    'Warehouse': {'A':4,'B':2},
    'A': {'C':5,'D':10},
    'B': {'A':3,'D':7},
    'C': {'D':4},
    'D': {}
}
```

3. Parcel Dataset

Code:

```
destinations = ['A','B','C','D']
parcels = pd.DataFrame({
    'Metric':[
        'Parcel_ID','Destination','Priority','Deadline'
    ],
    'Values':[
        [f'P{i}' for i in range(1,21)],
        [random.choice(destinations) for _ in range(20)],
        [random.randint(1,3) for _ in range(20)],
        [random.randint(5,20) for _ in range(20)]
    ]
})
parcels
```

Output:

	Metric	Values
0	Parcel_ID	[P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11,...
1	Destination	[C, D, C, A, A, D, B, B, D, D, C, D, B, A, B, ...
2	Priority	[1, 3, 1, 2, 3, 1, 2, 3, 1, 1, 1, 2, 2, 1, 3, ...
3	Deadline	[12, 20, 5, 13, 7, 14, 10, 19, 8, 20, 10, 14, ...

4. Algorithms

Code:

```
#Recursion-Route Cost
def route_cost(graph,start,end,visited=None):
    if visited is None:
        visited=set()
    if start==end:
        return 0
    visited.add(start)
    min_cost=float('inf')
    for neighbor,cost in graph[start].items():
        if neighbor not in visited:
            total=cost+route_cost(graph,neighbor,end,visited.copy())
            min_cost=min(min_cost,total)
    return min_cost
print("Cost from Warehouse to D:", route_cost(graph,'Warehouse','D'))

#Greedy-Parcel Selection
selected_parcels = parcels
print(selected_parcels)

#Dynamic Programming-Scheduling
deadlines=[random.randint(5,20) for _ in range(20)]
def check_schedule(deadlines):
    dp=[0]*(len(deadlines)+1)
    for i in range(1,len(deadlines)+1):
        dp[i]=max(dp[i-1], deadlines[i-1])
    return dp
check_schedule(deadlines)
```

Output:

```
Cost from Warehouse to D: 9
        Metric                                          Values
0    Parcel_ID  [P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11,...
1  Destination  [C, D, C, A, A, D, B, B, D, D, C, D, B, A, B, ...
2     Priority  [1, 3, 1, 2, 3, 1, 2, 3, 1, 1, 1, 2, 2, 1, 3, ...
3     Deadline  [12, 20, 5, 13, 7, 14, 10, 19, 8, 20, 10, 14, ...
[0,
 6,
 18,
 18,
 18,
 19,
 19,
 19,
 19,
 19,
 19,
 20,
 20,
 20,
 20,
 20,
 20,
 20,
 20,
 20,
 20]
```

5. Graph Algorithms

Code:

```
#Create Graph
G=nx.DiGraph()
for node in graph:
    for neighbor,weight in graph[node].items():
        G.add_edge(node,neighbor,weight=weight)

#Dijkstra Shortest Path
shortest_path = nx.dijkstra_path(G,'Warehouse','D')
print("Shortest Path:", shortest_path)

#Traveling Salesman Problem
nodes=['Warehouse','A','B','C','D']
def tsp(nodes):
    min_distance=float('inf')
    best_route=None
    for perm in itertools.permutations(nodes[1:]):
        route=['Warehouse']+list(perm)
        distance=0
        for i in range(len(route)-1):
            if route[i+1] in graph[route[i]]:
                distance+=graph[route[i]][route[i+1]]
            else:
                distance+=10
        if distance<min_distance:
            min_distance=distance
            best_route=route
    return best_route, min_distance
route, distance=tsp(nodes)
print("Optimal Route:",route)
print("Total Distance:",distance)
```

Output:

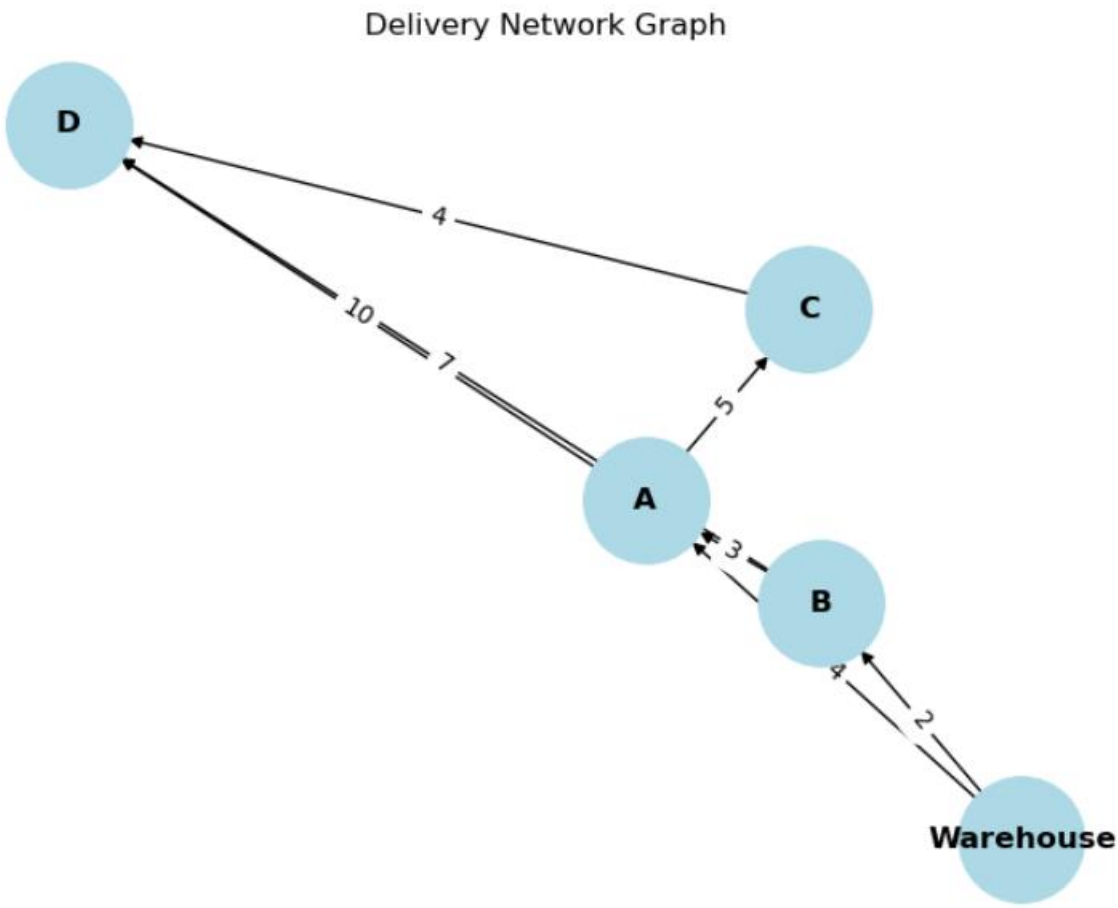
```
Shortest Path: ['Warehouse', 'B', 'D']
Optimal Route: ['Warehouse', 'B', 'A', 'C', 'D']
Total Distance: 14
```

6. Visualization

Code:

```
pos=nx.spring_layout(G)
nx.draw(G,pos,
        with_labels=True,
        node_size=2500,
        node_color='lightblue',
        font_weight='bold')
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
plt.title("Delivery Network Graph")
plt.show()
```

Output:



7. Performance Analysis

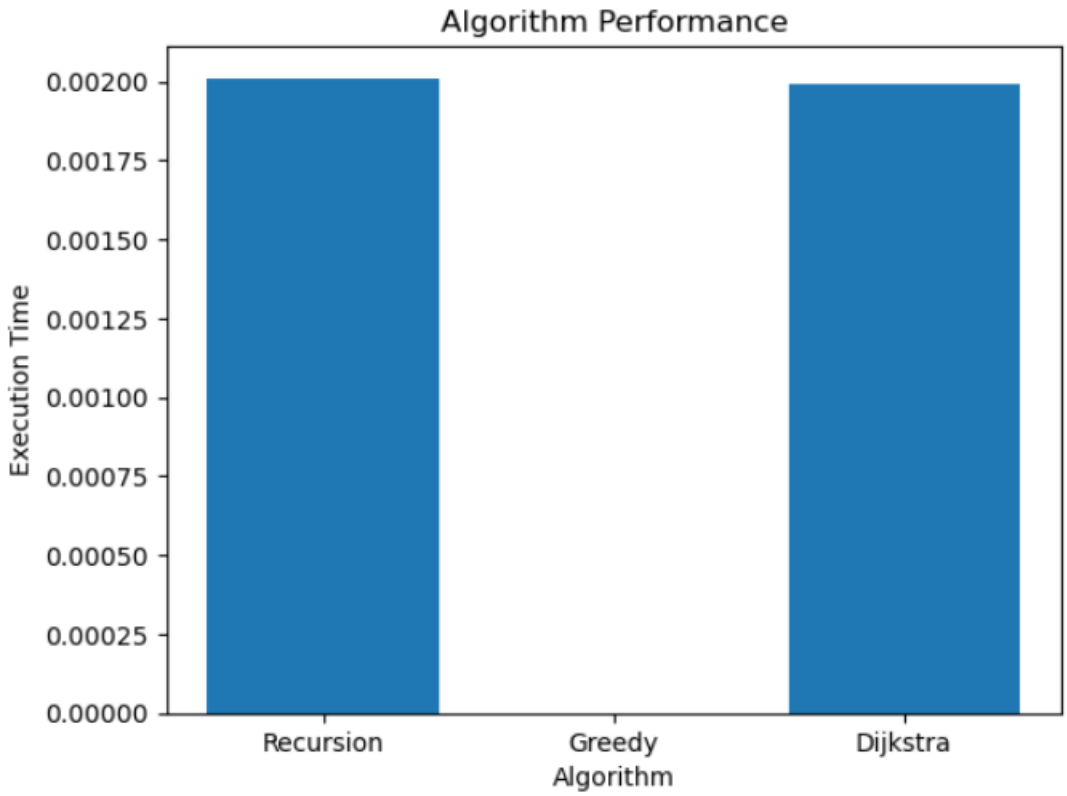
Code:

```
algorithms=["Recursion","Greedy","Dijkstra"]
execution_times=[]
start=time.time()
for _ in range(100):
    route_cost(graph,'Warehouse','D')
execution_times.append(time.time()-start)
start=time.time()
for _ in range(100):
    parcels
execution_times.append(time.time()-start)
start=time.time()
for _ in range(100):
    nx.dijkstra_path(G,'Warehouse','D')
execution_times.append(time.time()-start)
results=pd.DataFrame({
    "Algorithm":algorithms,
    "Execution Time":execution_times
})
print(results)

plt.bar(results["Algorithm"],results["Execution Time"])
plt.xlabel("Algorithm")
plt.ylabel("Execution Time")
plt.title("Algorithm Performance")
plt.show()
```

Output:

```
   Algorithm  Execution Time
0  Recursion        0.002011
1     Greedy        0.000000
2   Dijkstra        0.001993
```

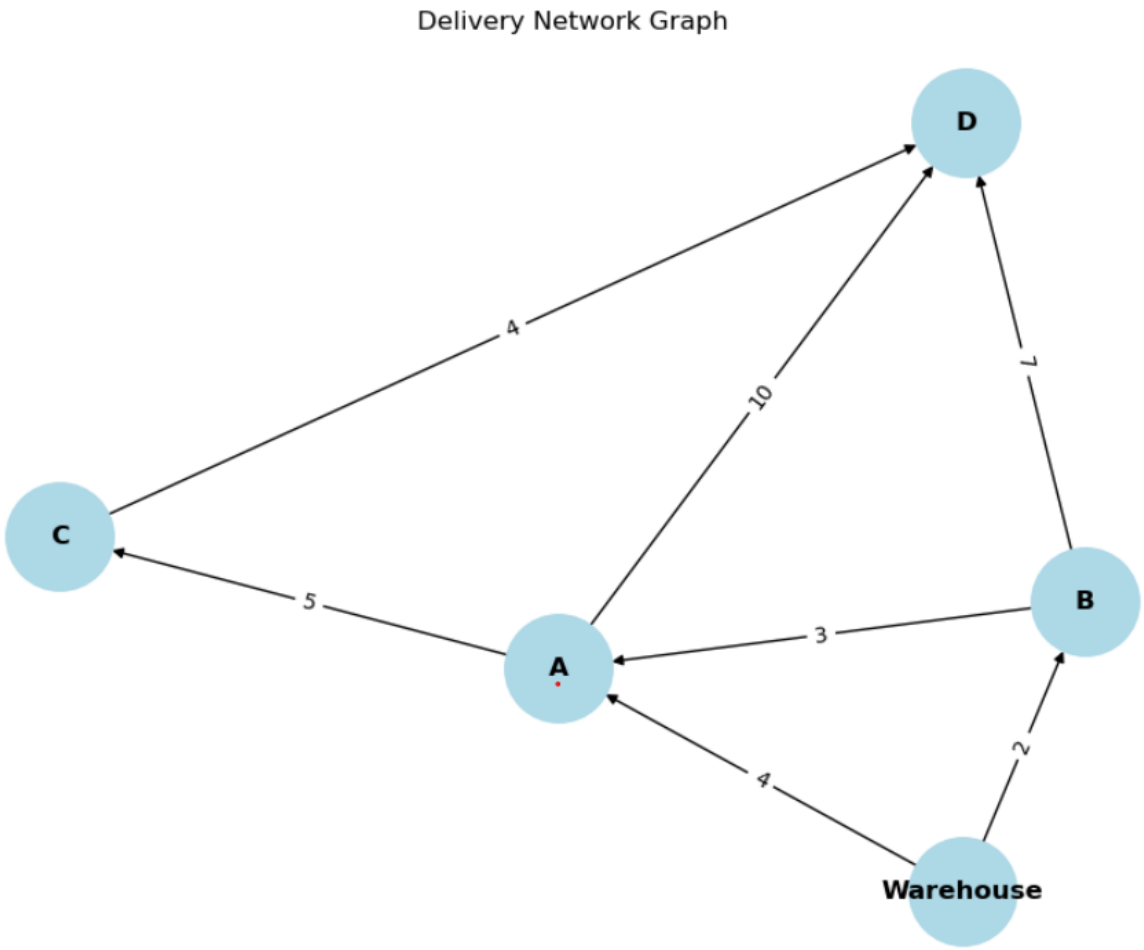


8. Route Simulation

Code:

```
pos=nx.spring_layout(G)
plt.figure(figsize=(8,6))
nx.draw(G,
        pos,
        with_labels=True,
        node_size=2500,
        node_color='lightblue',
        font_weight='bold')
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,
                             pos,
                             edge_labels=labels)
plt.title("Delivery Network Graph")
plt.show()
```

Output:



9. Final Summary

Code:

```
summary=pd.DataFrame({
    "Metric":[
        "Total Parcels",
        "Optimal Route",
        "Total Distance",
        "Algorithms Used"
    ],
    "Result":[
        20,
        str(route),
        distance,
        "Recursion,Greedy,DP,Dijkstra,TSP"
    ]
})
summary
```

Output:

	Metric	Result
0	Total Parcels	20
1	Optimal Route	['Warehouse', 'B', 'A', 'C', 'D']
2	Total Distance	14
3	Algorithms Used	Recursion,Greedy,DP,Dijkstra,TSP